## 1.Access one file from another

**Npm init –for creation of json file**

**Create index.js file**

```
1  const lovish = require("./second");
2
3  console.log("Hello world", lovish)
4
```

**Create second.js**

second.js > ...

```
1  harry = {
2      name: "Harry",
3      favNum: 36,
4      developer: true
5  }
6
7  module.exports = harry;
```

Handling HTTP request and response -Building HTTP Server in NodeJS

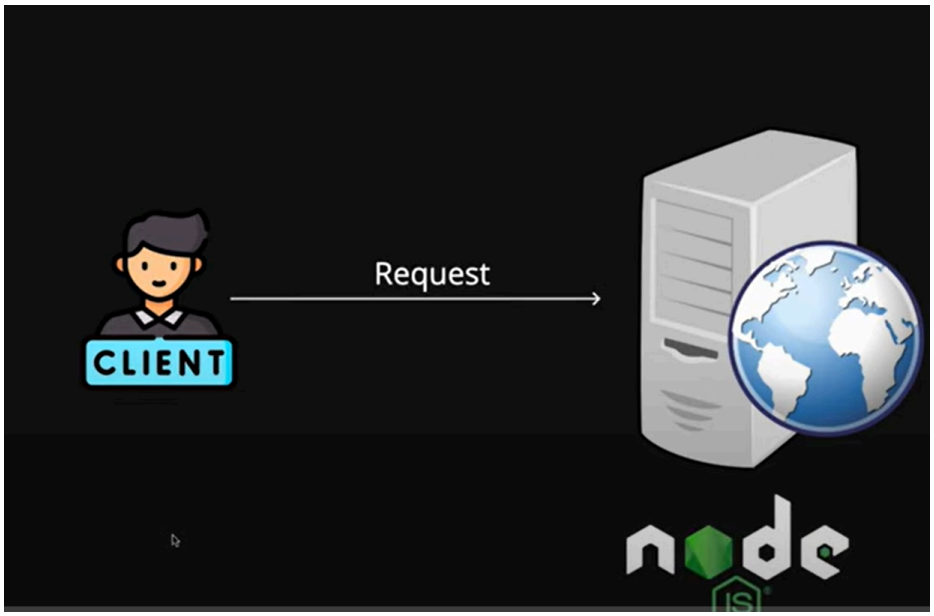
## Index.js

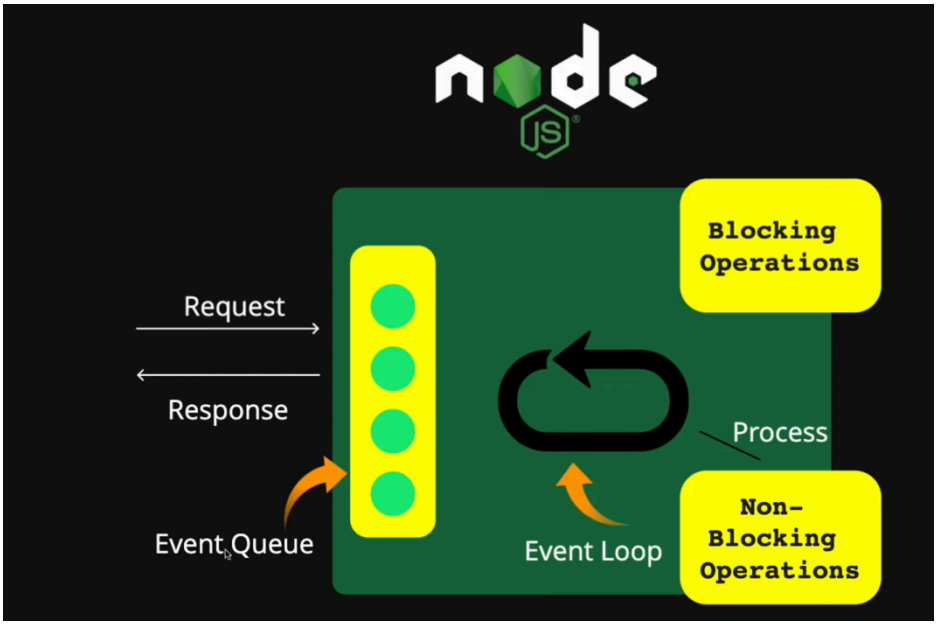
index.js > ...

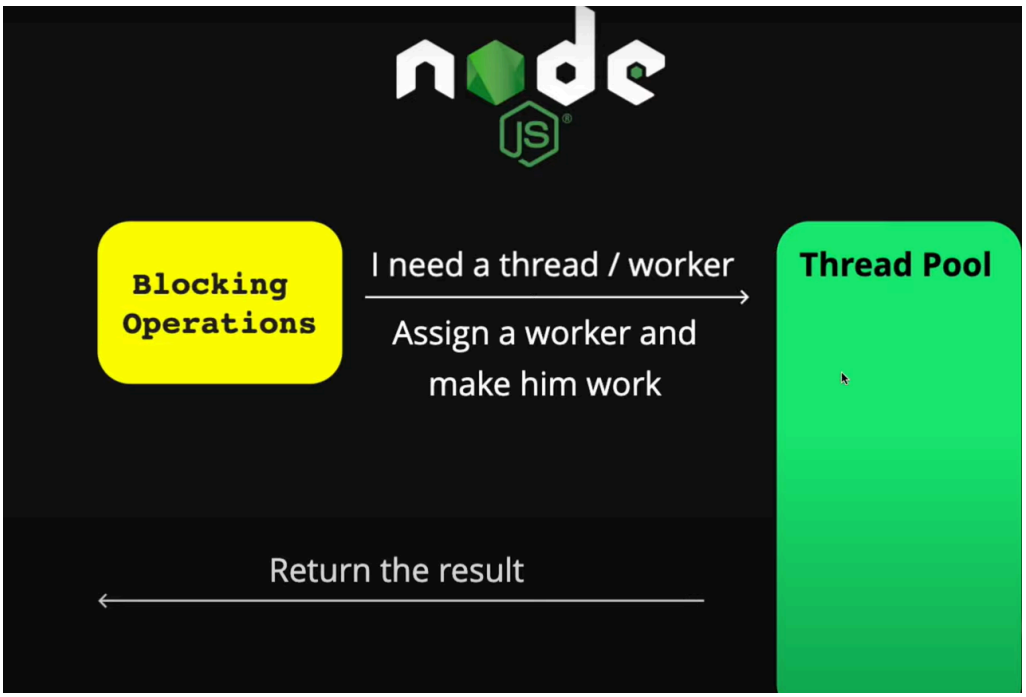
```
1  const http = require("http");
2
3  const myServer = http.createServer((req, res) => {
4    console.log("New Req Rec.");
5    res.end("Hello From Server");
6  });
7
8  myServer.listen(8000, () => console.log("Server Started!"));
```



# Architecture of Node.js






## For Example: Explanation (Not for Execution)

```
const fs = require("fs");

// Sync... Blocking....
fs.writeFileSync("./test.txt", "Hello World");

// Async.. Non - Blocking Req
fs.writeFile("./test.txt", "Hello World Async", (err) => {})
```

## Blocking example:

```
file.js > ...
2
3   console.log("1");
4   // Blocking...
5   const result = fs.readFileSync("contacts.txt", "utf-8");
6   console.log(result);
7
8   console.log("2");
```

**Shows output :**

**1**

**Result from file contacts.txt**

**2**

**Non-blocking Example:**




**Output:**

**1**

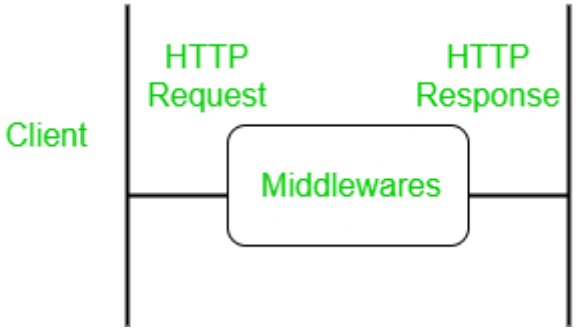
**2**

**Result from contacts.txt**

# Middleware in Express

- **Express** serves as a **routing and Middleware framework** for handling the different routing of the webpage and it works between the request and response cycle.
- Middleware gets executed after the server receives the request and before the controller actions send the response. Middleware has **and** access to the request object, responses object, and next, it can process the request before the server sends a response. An Express-based application is a series of

middleware function calls. In this article, we will discuss what is middleware in express.js.




*Middleware working*

# What is Middleware in Express JS?

Middleware is a request handler that allows you to intercept and manipulate requests and responses before they reach route handlers. They are the functions that are invoked by the Express.js routing layer.
It is a flexible tool that helps in adding functionalities like logging, authentication, error handling, and more to Express applications.

**Syntax:**

The basic syntax for the middleware functions is:

```
app.get(path, (req, res, next) => {}, (req, res) => {})
```

Middleware functions take 3 arguments: the **request object**, the **response object**, and the **next function** in the application's request-response cycle, i.e., two objects and one function.

- Middleware functions execute some code that can have side effects on the app and usually add information to the request or response objects. They are also capable of ending the cycle by sending a response when some condition is satisfied. If they don't send the response when they are done, they start the execution of the **next function** in the stack. This triggers calling the 3rd argument, next().
- The middle part **(req,res,next)=>{}** is the middleware function. Here we generally perform the actions required before the user is allowed to view the webpage or call the data and many other functions. So let us create our own middleware and see its uses.
  - Express.js, a popular web framework for NodeJS, heavily utilizes middleware for request processing. Middleware functions in Express.js can be added using the app.use() method or specific HTTP method functions such as app.get(), app.post(), etc., to specify middleware for specific routes.

# Types of Middleware

Express JS offers different types of middleware and you should choose the middleware on the basis of functionality required.

- **Application-level middleware:** Bound to the entire application using **app.use()** or **app.METHOD()** and executes for all routes.

- **Router-level middleware**: Associated with specific routes using **router.use()** or **router.METHOD()** and executes for routes defined within that router.
- **Error-handling middleware:** Handles errors during the request-response cycle. Defined with four parameters (err, req, res, next).
- **Built-in middleware:** Provided by Express (e.g., express.static, express.json, etc.).
- **Third-party middleware**: Developed by external packages (e.g., body-parser, morgan, etc.).

# Steps to Implement Middleware in Express

**Step 1:** Go to your project directory and enter the following command to create a NodeJs project. Make sure that NodeJs is installed in your machine.

```
npm init -y
```

**Step 2:** Install two dependencies using the following command.

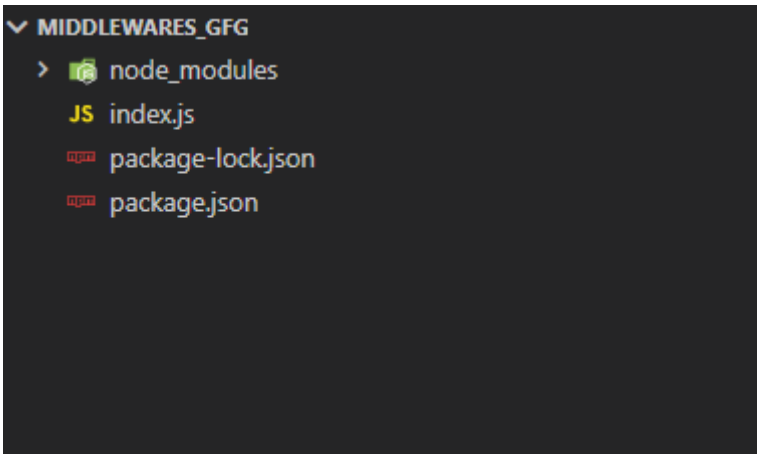
```
npm install express nodemon
```

**Step 3:** In the scripts section of the package.json file, add the following code line.

```
"start": "nodemon index.js",
```

**Step 4:** Create an index.js file in the directory. Make sure that it is not inside any subdirectories of the directory you are working in.

**Project Structure:**




*Project Structure*

The updated dependencies in **package.json** file will look like**:**

```
"dependencies": {
   "express": "^4.19.2",
   "nodemon": "^3.1.4"
}
```

**Step 5:** Now we will set up our **express app** and send a **response** to our **server**. Here is the code for the **index.js** file.

JavaScript

```
// Filename - index.js

const express = require("express");
const app = express();
```

```
const port = process.env.port || 3000;
app.get("/", (req, res) => {
   res.send(`<div>
   <h2>Welcome to GeeksforGeeks</h2>
   <h5>Tutorial on Middleware</h5>
 </div>`);
});
app.listen(port, () => {
   console.log(`Listening to port ${port}`);
});
```

**Step to run the application:** Run the code by entering the following command on the terminal.

```
npm start
```

**Output:**




**Example 1:** Creating a Middleware in the app.get() function, modify accordingly to the following code.

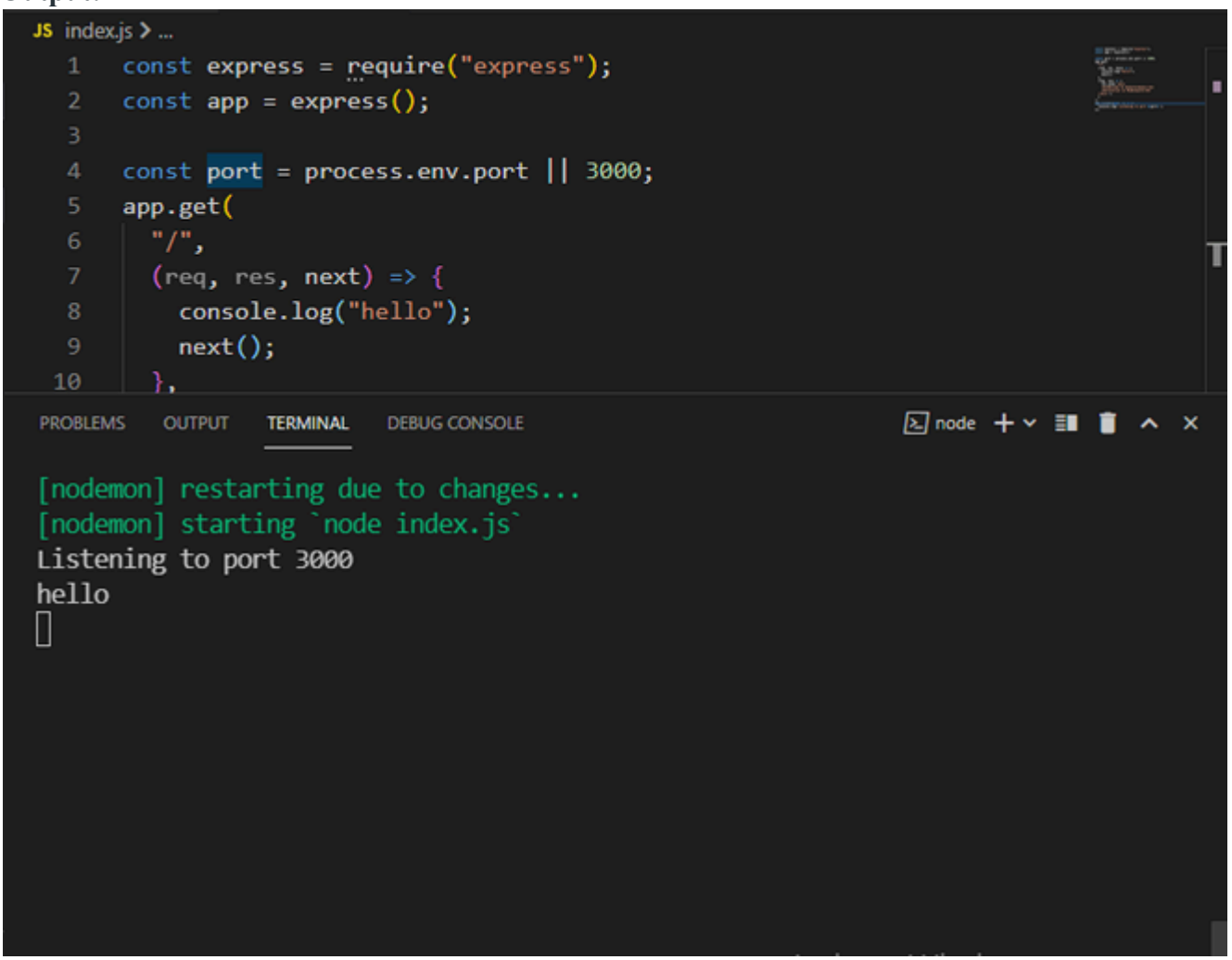
JavaScript

```
// Filename - index.js

app.get(
   "/",
   (req, res, next) => {
      console.log("hello");
      next();
   },
   (req, res) => {
      res.send(
         `<div>
            <h2>Welcome to GeeksforGeeks</h2>
            <h5>Tutorial on Middleware</h5>
         </div>`
      );
```

```
  }
);
```

**Output:**


JS index.js > ...
1 const express = require("express");
2 const app = express();
3
4 const port = process.env.port || 3000;
5 app.get(
6 "/",
7 (req, res, next) => {
8 console.log("hello");
9 next();
10 },
PROBLEMS
OUTPUT
TERMINAL
DEBUG CONSOLE
node
[nodemon] restarting due to changes...
[nodemon] starting `node index.js`
Listening to port 3000
hello